COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S1
DATE PERFORMED:	08/04/2026	**DATE SUBMITTED:**	08/11/26
NAME:	Pangilinan Jr., Bienvenido B.	**INSTRUCTOR:**	Engr. Jimlord Quehado

ACTIVITY NO. 5 **[ACTIVITY NAME] Queues**

1. PROCEDURE OUTPUT

What procedures did you perform?

ILOA

Source Code

```
#include <iostream>
#include <queue>

void display(std::queue<char> r);

int main() {

   std::queue<char> q;

   q.push('c');
   q.push('p');
   q.push('e');
   q.push('0');
   q.push('1');
   q.push('0');

   //display
   display(q);

   //empty()
   std::cout << "Is it empty? " << (q.empty() ? "Yes" : "No") << "\n";

   //size()
   std::cout << "Size: " << q.size() << "\n";

   //front
   std::cout << "Front: " << q.front() << "\n";

   //back
   std::cout << "Back: " << q.back() << "\n";

   //pop
   q.pop();
   std::cout << "Popped element.\n";
```

```
    //display
    std::cout << "After popping the element: ";
    display(q);

    //push another element
    q.push('B');
    std::cout << "Pushed into the queue.\n";

    display(q);

    //deleting all element
    while (!q.empty()) {
        q.pop();
    }
    std::cout << "all elements are deleted.\n";

}

void display(std::queue<char> r) {
    std::queue <char> c = r;

    while (!c.empty()) {
        std::cout << c.front() << " ";
        c.pop();
    }
    std::cout << "\n";
}
```

Output

```
c p e 0 1 0
Is it empty? No
Size: 6
Front: c
Back: 0
Popped element.
After popping the element: p e 0 1 0
Pushed into the queue.
p e 0 1 0 B
all elements are deleted.

Process finished with exit code 0
```

Analysis:

In this activity it show how a queue works using the First-In, First-Out (FIFO) rule, to show the First In First Out (FIFO) rule, by placing 6 characters (c p e 0 1 0), checking the **size**, the **front** and **back** items, **popping** the first character ('c'), pushing a new item ('B') to the back, and finally using a **while** loop to delete every other remaining element until the queue is empty.

ILOB

Source Code

```
#include <iostream>
#include "queuell.h"

int main() {
   qNode<char>* front = nullptr;
   qNode<char>* back = nullptr;

   enqueue(&front, &back, 'J');
   std::cout << front->data << " " << back->data << std::endl;
   enqueue(&front, &back, 'I');
   std::cout << front->data << " " << back->data << std::endl;
   enqueue(&front, &back, 'M');
   std::cout << front->data << " " << back->data << std::endl;
   dequeue(&front, &back);
   std::cout << front->data << " " << back->data << std::endl;

   return 0;
}
```

Output

```
J J
J I
J M
I M


Process finished with exit code 0
```

Analysis:

In this process the program starts to push the ('J', 'I'), and ('M') one by one onto the queue, now printing the front and back values printing the front and back values each time. Finally, it calls **dequeue** to remove the leading element 'J'. Now **front** will point to 'I' while **back** will still point to 'M'.

Source Code

```
#include <iostream>
#include "queuell.h"

int main() {
   qNode<char>* front = nullptr;
   qNode<char>* back = nullptr;

   enqueue(&front, &back, 'B');
   enqueue(&front, &back, 'I');
   enqueue(&front, &back, 'E');
   enqueue(&front, &back, 'N');
   display(front);
   dequeue(&front, &back);
   display(front);

   std::cout << Front(&front);

   return 0;
}
```

Output

```
B I E N
I E N
I
Process finished with exit code 0
```

Analysis:
In this part, you will modify the main file while adding new functions in the header file which is to display all elements in the list, it displays the list in horizontal line then once it got called the next function which is **dequeue** or to remove an element, it removes the head ('B') then proceed to display the remaining element ('I', 'E', 'N'), after removing the front element, then the **front** function was called, making the console display only the first or front element which is ('I')

ILOB.2

Source Code

```
#include <iostream>
#include <string>
#include "queueArr.h"

int main() {
```

```
    queueArr<std::string> q1(10);

    // Empty()
    std::cout << "\nIs q1 empty? " << (q1.Empty() ? "Yes" : "No") <<
std::endl;

    // Enqueue()
    q1.Enqueue("B");
    q1.Enqueue("I");
    q1.Enqueue("E");
    q1.Enqueue("N");
    q1.Enqueue("1");
    q1.Enqueue("7");

    // Size()
    std::cout << "\nCurrent size of q1: " << q1.Size() << std::endl;

    // display all
    std::cout << "\nQueue contents: ";
    display(q1);

    // Front()
    std::cout << "\nFront element: " << q1.Front() << std::endl;

    // Back()
    std::cout << "\nBack element: " << q1.Back() << std::endl;

    // Dequeue()
    q1.Dequeue();
    std::cout << "\nDequeue: " << q1.Front() << std::endl;

    // Copy Constructor
    queueArr<std::string> q2 = q1;
    std::cout << "\nq2 (Copied): ";
    display(q2);

    // Copy Assignment Operator
    queueArr<std::string> q3(10);
    q3 = q1;
    std::cout << "\nq3 (Assigned): ";
    display(q3);

    // Clear()
    q1.Clear();
    std::cout << "\nq1 cleared. Is q1 empty now? " << (q1.Empty() ? "Yes" :
"No") << std::endl;

    return 0;
}
```

Output

```
Queue created with capacity: 10

Is q1 empty? Yes

Current size of q1: 6

Queue contents: Queue elements: B I E N 1 7
Queue destroyed.

Front element: B

Back element: 7

Dequeue: I

q2 (Copied): Queue elements: I E N 1 7
Queue destroyed.
Queue created with capacity: 10

q3 (Assigned): Queue elements: I E N 1 7
Queue destroyed.

q1 cleared. Is q1 empty now? Yes
Queue destroyed.
Queue destroyed.
Queue destroyed.

Process finished with exit code 0
```

Analysis:
Through this activity, this code demonstrates on how the **queue Array** works. At first by putting your capacity variable in 10, it begins to validating the q1 is **empty** then then **enqueue** initialize by sending the six strings in the main file (B, I, E, N, 1, 7) to bring a total of six string size, and then after identifying the **front** variable (B) and **back** variable (7), after it, it executes the **dequeue** process to remove the leading element, shifting the front to

(I), then the program run to next one which is by copying by creating the q2 using using the **copy constructor** and assigning q1 data to q3 using the **copy assignment operator,** now with displaying the (I E N 1 7), it Finally calls to **clear** the rest and the **destructor**, making the output in the last part that the queue is already empty and destroyed.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

ILOC

Source Code

```
#include <iostream>
#include "Job.h"
#include "Printer.h"

int main() {
   Printer officePrinter;

   std::cout << "--- OFFICE PRINTER ---\n";

   // First employees to send files in the printer
   std::cout << "Submitting the first files:\n";
   officePrinter.addJob(Job(1, "Bien", 5));
   officePrinter.addJob(Job(2, "Jhum", 12));
   officePrinter.addJob(Job(3, "Dave", 3));

   officePrinter.displayQueue();

   //Processing the upcoming first job
   std::cout << "\nProcessing first job:\n";
   officePrinter.processNextJob();

   //New submissions while printer is busy
   std::cout << "\nSubmitting New Files:\n";
   officePrinter.addJob(Job(4, "Allen", 8));
   officePrinter.addJob(Job(5, "Jenuel", 15));
   officePrinter.addJob(Job(6, "Jayne", 4));

   officePrinter.displayQueue();

   //Completing all pending files
   std::cout << "\nProcessing current queue:\n";
   officePrinter.processAllJobs();

   //Late upcoming files
   std::cout << "\nSubmitting Late Files:\n";
   officePrinter.addJob(Job(7, "Mia", 10));
   officePrinter.addJob(Job(8, "Drens", 2));
   officePrinter.addJob(Job(9, "Renz", 7));
   officePrinter.addJob(Job(10, "Ali", 6));
```

```
    officePrinter.processAllJobs();

    return 0;
}
```

Output

```
--- OFFICE PRINTER ---
Submitting the first files:
(ADDED) [Job ID: 1 | User: Bien | Pages: 5] added to print queue.
(ADDED) [Job ID: 2 | User: Jhum | Pages: 12] added to print queue.
(ADDED) [Job ID: 3 | User: Dave | Pages: 3] added to print queue.
(STATUS) Pending Jobs in Queue (3/10):
  1. [Job ID: 1 | User: Bien | Pages: 5]
  2. [Job ID: 2 | User: Jhum | Pages: 12]
  3. [Job ID: 3 | User: Dave | Pages: 3]

Processing first job:
[PRINTING] Currently processing: [Job ID: 1 | User: Bien | Pages: 5] ... Finished!

Submitting New Files:
(ADDED) [Job ID: 4 | User: Allen | Pages: 8] added to print queue.
(ADDED) [Job ID: 5 | User: Jenuel | Pages: 15] added to print queue.
(ADDED) [Job ID: 6 | User: Jayne | Pages: 4] added to print queue.
(STATUS) Pending Jobs in Queue (5/10):
  1. [Job ID: 2 | User: Jhum | Pages: 12]
  2. [Job ID: 3 | User: Dave | Pages: 3]
  3. [Job ID: 4 | User: Allen | Pages: 8]
  4. [Job ID: 5 | User: Jenuel | Pages: 15]
  5. [Job ID: 6 | User: Jayne | Pages: 4]

Processing current queue:

--- Starting Batch Print Processing ---
[PRINTING] Currently processing: [Job ID: 2 | User: Jhum | Pages: 12] ... Finished!
[PRINTING] Currently processing: [Job ID: 3 | User: Dave | Pages: 3] ... Finished!
[PRINTING] Currently processing: [Job ID: 4 | User: Allen | Pages: 8] ... Finished!
[PRINTING] Currently processing: [Job ID: 5 | User: Jenuel | Pages: 15] ... Finished!
[PRINTING] Currently processing: [Job ID: 6 | User: Jayne | Pages: 4] ... Finished!
--- All jobs completed. ---

Submitting Late Files:
(ADDED) [Job ID: 7 | User: Mia | Pages: 10] added to print queue.
(ADDED) [Job ID: 8 | User: Drens | Pages: 2] added to print queue.
(ADDED) [Job ID: 9 | User: Renz | Pages: 7] added to print queue.
(ADDED) [Job ID: 10 | User: Ali | Pages: 6] added to print queue.

--- Starting Batch Print Processing ---
[PRINTING] Currently processing: [Job ID: 7 | User: Mia | Pages: 10] ... Finished!
[PRINTING] Currently processing: [Job ID: 8 | User: Drens | Pages: 2] ... Finished!
[PRINTING] Currently processing: [Job ID: 9 | User: Renz | Pages: 7] ... Finished!
[PRINTING] Currently processing: [Job ID: 10 | User: Ali | Pages: 6] ... Finished!
--- All jobs completed. ---

Process finished with exit code 0
```

Analysis:
In this activity the program uses the FIFO rule in it to make a simulation of an office printer, now the first three employees, bien, jhum, and dave to send their files to the printer to print them, it uses the **processNextJob()** to print bien's 5-page document before anyone else. Jhum and Dave are waiting in line, and three more files are added to the back of the queue which are allen, jenuel, and jayne, now there are now five print tasks pending. The program calls **processAllJobs()** which prints and removes each remaining document in strict chronological order until the queue is empty, after that there is late batch of four files from Mia, Drens, Renz, and Ali is enqueued and immediately cleared in a final batch run.

Defending:
I chose an array-based implementation for the queue because of its memory efficiency and fast indexed access. Since arrays store elements in contiguous memory locations, they offer better cache locality and performance compared to linked lists, which require extra memory overhead for storing node pointers.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.